Diffusion Models

from intuition and DDPM mathematics to conditional, latent, fast, and applied diffusion

Miodrag Bolic

University of Ottawa

March 18, 2026

Outline: Diffusion Models

- **Story and score-based viewpoint**
  - What a diffusion model is intended to do
  - Scores, denoising score matching, and Langevin intuition
- **DDPM fundamentals**
  - Forward noising process
  - Noise-prediction / denoising objective
  - Reverse sampling intuition and algorithmic steps
- **Extensions and applications**
  - Conditional diffusion, latent diffusion, and fast samplers
  - Signals / time series and structured prediction algorithms
  - Vision applications, recent trends, and study questions

Notebook: `Diffusion_Session_Notebook.ipynb`

Why do diffusion models fit this course?

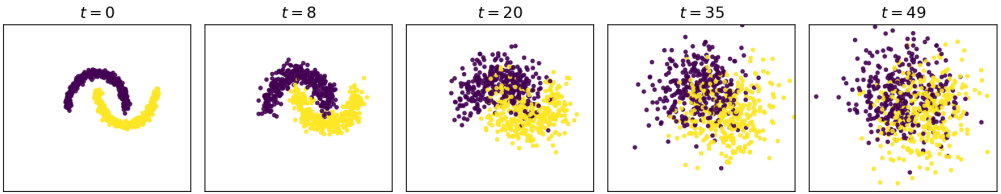
- Diffusion models learn a **full probability distribution**, not only a point estimate.
- Sampling is central: uncertainty is represented through **multiple plausible draws**.
- The framework connects three ideas that are already natural in this course:
  - Controlled noisy perturbations,
  - Gradients of log-density (**scores**),
  - Stochastic simulation.

Main idea

Instead of learning generation in one hard step, diffusion learns to **undo many easy Gaussian corruption steps**.

Diffusion models learn how to reverse a simple corruption process

- Start with a clean sample x_0.
- Corrupt it gradually with small amounts of noise until the sample is nearly Gaussian.
- Train a network that knows the noise level and learns how to undo one corruption step.
- At test time, begin from pure noise and repeatedly apply the learned reverse step to generate a sample.



Session notebook figure: a simple two-moons dataset progressively loses structure as the noise level increases.

Score functions and Langevin Monte Carlo

Score of a density

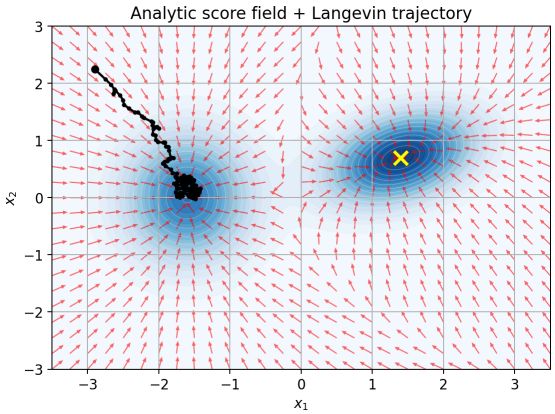
$$\nabla_{\boldsymbol{x}} \log p(\boldsymbol{x})$$

points toward directions of increasing log-density.

Langevin update

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \eta \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}_k) + \sqrt{2\eta}\, \boldsymbol{z}_k, \boldsymbol{z}_k \sim \mathcal{N}(0, I)$$

- Deterministic drift moves samples toward high-density regions,
- Injected noise preserves stochastic exploration,
- Annealed Langevin uses several noise levels.



From score matching to denoising score matching

Problem: we do not know $\nabla_x \log p_{\text{data}}(x)$ directly.

Score matching idea: fit a neural network $s_\theta(x)$ to approximate the score field.

Denoising score matching intuition

Corrupt clean data by Gaussian noise,

$$\tilde{x} = x + \sigma\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I),$$

and train a network to predict how noisy points should move back toward likely clean points.

For Gaussian perturbations, the optimal denoiser is directly related to the score of the corrupted density:

$$\nabla_{\tilde{x}} \log p_\sigma(\tilde{x}) \propto \mathbb{E}[x \mid \tilde{x}] - \tilde{x}.$$

This is the conceptual bridge from **score-based models** to **diffusion denoisers**.

Forward diffusion process

A DDPM defines a Markov chain that gradually destroys structure:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\left(\sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\right).$$

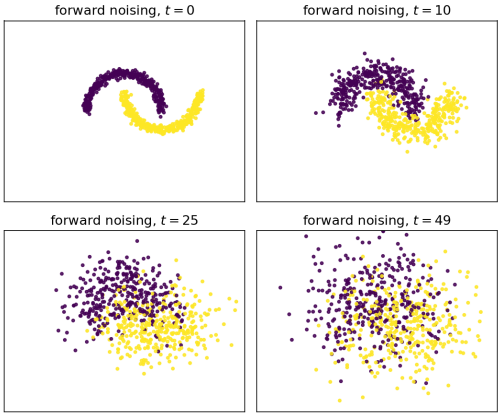
Define

$$\alpha_t = 1 - \beta_t, \qquad \bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s.$$

Then we can sample any noise level in one step:

$$q(x_t \mid x_0) = \mathcal{N}\left(\sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t) I\right).$$

- early t: data structure remains visible,
- late t: distribution becomes almost isotropic Gaussian.



Session notebook figure: the two-moons structure gradually collapses into a near-Gaussian cloud.

The DDPM training objective

Using the closed-form forward process,

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}, \qquad \boldsymbol{\epsilon} \sim \mathcal{N}(0, I).$$

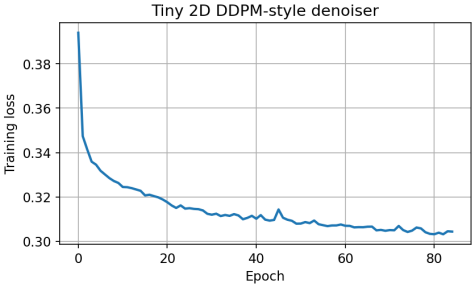
A network $\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t)$ is trained to predict the injected noise:

$$\mathcal{L}_{\text{simple}}(\theta) = \mathbb{E}_{t,\boldsymbol{x}_0,\boldsymbol{\epsilon}}\left[\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t)\|_2^2\right].$$

Why this works

Predicting noise, estimating the clean sample $\boldsymbol{x}_0$, and learning the score field are closely related parameterizations of the same denoising problem.

Common parameterizations: ϵ-**prediction** (predict the added noise), x_0-**prediction** (predict the clean sample), v-**prediction** (an alternative reparameterization).



Small CPU demo from the notebook: even a tiny 2D denoiser learns a useful noise-prediction objective.

Variational viewpoint: from ELBO to the simple loss

The original DDPM derivation optimizes a variational lower bound (ELBO):

$$\mathcal{L}_{\mathrm{VLB}} = D_{\mathrm{KL}}\big(q(\boldsymbol{x}_T \mid \boldsymbol{x}_0) \,\|\, p(\boldsymbol{x}_T)\big) + \sum_{t=2}^{T} D_{\mathrm{KL}}\big(q(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t, \boldsymbol{x}_0) \,\|\, p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t)\big) - \log p_\theta(\boldsymbol{x}_0 \mid \boldsymbol{x}_1).$$

- Each timestep contributes a denoising / matching term.
- With common parameter choices, this simplifies to the familiar MSE noise-prediction objective.
- Graduate-level message: the practical DDPM loss is a computationally convenient surrogate of a principled variational objective.

This is why DDPM can be presented consistently as both a **latent-variable model** and a **score-based denoiser**.

Reverse sampling intuition

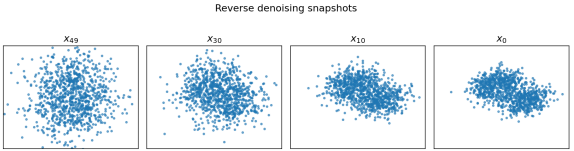
The reverse chain is learned as

$$p_\theta(\boldsymbol{x}_{t-1} \mid \boldsymbol{x}_t) = \mathcal{N}\big(\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t),\, \Sigma_t\big).$$

With ϵ-prediction, a standard ancestral update is ($\boldsymbol{z} \sim \mathcal{N}(0, I)$)

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t) \right) + \sigma_t \boldsymbol{z}.$$

- The mean moves us toward a cleaner sample,
- The Gaussian term keeps the chain stochastic,
- Repeated small denoising steps transform noise into data.



Session notebook figure: snapshots from a tiny two-moons reverse sampler.

Algorithm 1: DDPM training and inference

Training

1. Sample a clean datum $\boldsymbol{x}_0$.
2. Sample a timestep $t \in \{1, \ldots, T\}$.
3. Sample $\boldsymbol{\epsilon} \sim \mathcal{N}(0, I)$ and form
$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\, \boldsymbol{x}_0 + \sqrt{1 - \bar{\alpha}_t}\, \boldsymbol{\epsilon}.$$
4. Evaluate the denoiser $\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t)$.
5. Minimize $\|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t)\|_2^2$ and update θ.

Inference

1. Initialize $\boldsymbol{x}_T \sim \mathcal{N}(0, I)$.
2. For $t = T, T-1, \ldots, 1$:
   - predict $\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t)$,
   - compute the reverse mean,
   - optionally add Gaussian noise.
3. Return the final sample $\boldsymbol{x}_0$.

Interpretation

Training sees random noise levels in parallel. Inference removes noise sequentially, one reverse step at a time.

How does the SDE view connect?

The ordinary/stochastic differential equation (ODE/SDE) view gives the continuous-time picture.

- **score-based models**, and
- **diffusion through differential equations**

are not competing ideas; they are two views of the same family.

Continuous-time VP-SDE

$$d\boldsymbol{x} = -\tfrac{1}{2}\beta(t)\boldsymbol{x}\,dt + \sqrt{\beta(t)}\,d\boldsymbol{w}_t.$$

The reverse-time SDE uses the score:

$$d\boldsymbol{x} = \Big[-\tfrac{1}{2}\beta(t)\boldsymbol{x} - \beta(t)\nabla_{\boldsymbol{x}}\log p_t(\boldsymbol{x})\Big]\,dt + \sqrt{\beta(t)}\,d\bar{\boldsymbol{w}}_t.$$

DDPM can be viewed as a **discrete approximation** of this broader score-based diffusion framework.

Conditional diffusion

We often want to generate samples conditioned on side information:

$$p_\theta(x \mid c)$$

where the condition c may represent:

- a class label,
- a text prompt,
- observed pixels or a mask,
- past values in a time series,
- a measurement in an inverse problem.

Training idea: provide the condition to the denoiser,

$$\epsilon_\theta(x_t, t, c).$$

The model still learns to remove noise, but now it does so **in a way that is consistent with the condition**.

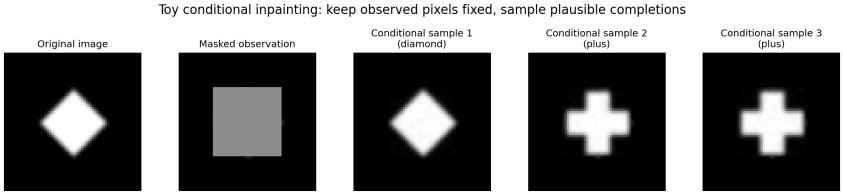
Conditional diffusion: a vision example

Example: image inpainting

Given a partially observed image y and a mask M, we want to sample a plausible completion:

$$p_\theta(\boldsymbol{x} \mid y, M).$$

- The visible pixels are kept fixed.
- The diffusion model fills in the missing region.
- Different reverse trajectories can produce different plausible completions.



Notebook figure: original, masked observation, and conditional completions.

Algorithm 2: conditional diffusion in practice

Training

1. Sample a clean target $\boldsymbol{x}_0$ and condition c.
2. Sample timestep t and noise $\boldsymbol{\epsilon}$.
3. Form $\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon}$.
4. Predict $\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t, c)$.
5. Optimize the conditional denoising loss.

Typical conditions c: text, class labels, masks, observed pixels, past signal values, or measurements.

Inference

1. Initialize the unknown part from noise.
2. For $t = T, \ldots, 1$, denoise using the condition c.
3. Re-impose known pixels / observed context after each step when required.
4. Draw multiple samples to represent the conditional uncertainty.

Application view

Inpainting keeps visible pixels fixed; forecasting keeps the observed past fixed while sampling multiple plausible futures.

Guidance: steering the reverse chain

Classifier guidance: modify the score using a classifier gradient,

$$\nabla_{x_t} \log p(x_t \mid c) = \nabla_{x_t} \log p(x_t) + \nabla_{x_t} \log p(c \mid x_t).$$

Classifier-free guidance (CFG): train conditional and unconditional denoisers together and combine them at sampling time:

$$\epsilon_{\text{cfg}}(x_t, t, c) = \epsilon_\theta(x_t, t, \varnothing) + w\Big(\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \varnothing)\Big).$$

- larger w usually enforces the condition more strongly,
- but can reduce diversity and sometimes harm realism.

This is a useful uncertainty lesson: **stronger conditioning can trade off against calibrated sample diversity**.

Diffusion for signals and time series

For signals, we typically condition on context:

$$p_\theta(x_{\text{future}} \mid x_{\text{past}}, c)$$

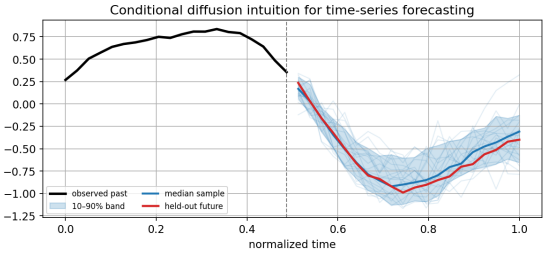
or on an observation mask for imputation.

What changes relative to images?

- Architectures become 1D CNNs, temporal U-Nets, or Transformers,
- Conditions include timestamps, masks, covariates, and known exogenous inputs,
- Evaluation should focus on **probabilistic forecasts**, not only point accuracy.

Representative ideas:

- **TimeGrad**,
- **CSDI**.



Latent diffusion

Pixel-space diffusion is powerful but expensive.

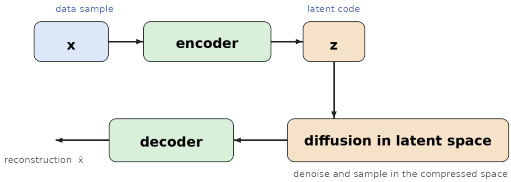
Idea

1. train an autoencoder $x \leftrightarrow z$,
2. run diffusion in latent space z,
3. decode the final latent back to data space.

Benefits

- fewer dimensions,
- faster training and sampling,
- easier high-resolution conditioning.

This is the key idea behind modern large-scale text-to-image systems.



Algorithm 3: latent diffusion training and inference

Training

1. Encode data to a latent code $z_0 = E(x)$.
2. Sample timestep t and latent noise ϵ.
3. Form a noisy latent z_t.
4. Train a latent denoiser $\epsilon_\theta(z_t, t, c)$.
5. Keep the autoencoder fixed or jointly fine-tune, depending on the system.

Inference

1. Sample $z_T \sim \mathcal{N}(0, I)$.
2. Run reverse diffusion in latent space.
3. Decode the final latent sample: $\hat{x} = D(z_0)$.

Why it helps

The expensive diffusion steps happen in a lower-dimensional latent space, which reduces cost while keeping high-resolution generation feasible.

Fast sampling and sample-based evaluation

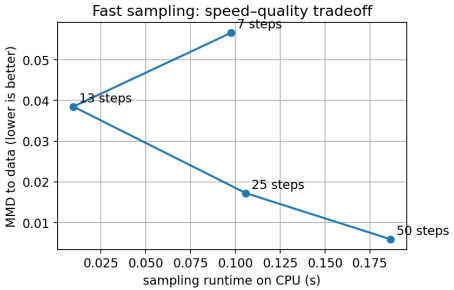
Naive ancestral DDPM sampling may use hundreds or thousands of reverse steps.

Popular acceleration ideas:

- **DDIM**: deterministic or semi-deterministic non-Markovian sampling,
- **DPM-Solver**: higher-order ODE solvers,
- **Consistency / shortcut-style models**: learn to jump across many steps.

Evaluation lesson for this course

- image generation: FID, coverage / diversity, human inspection,
- time series and signals: CRPS, interval coverage, sampled forecast quality.



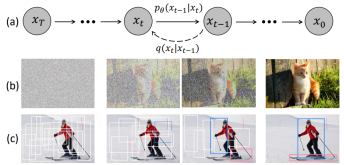
Application section: why diffusion matters in practice

Beyond image generation, diffusion models are attractive because they support **sampling-based uncertainty** in many tasks:

- **forecasting and imputation:** multiple plausible futures or fills-in for missing data,
- **inverse problems:** posterior samples consistent with measurements,
- **high-resolution synthesis/editing:** flexible conditional generation in latent space,
- **scientific and engineering measurements:** reconstruction with uncertainty, not only point estimates.

Top-conference anchors for this section: TimeGrad (ICML 2021), CSDI (NeurIPS 2021), latent diffusion (CVPR 2022), DiffusionDet (ICCV 2023), ODISE (CVPR 2023), and Marigold (CVPR 2024).

Application I: vision tasks beyond image synthesis



Object detection

DiffusionDet (ICCV 2023): denoises noisy bounding boxes into final object detections.

Segmentation

ODISE (CVPR 2023): diffusion representations for open-vocabulary panoptic segmentation.

Depth

Marigold (CVPR 2024): repurposes a pretrained latent diffusion model for monocular depth estimation.

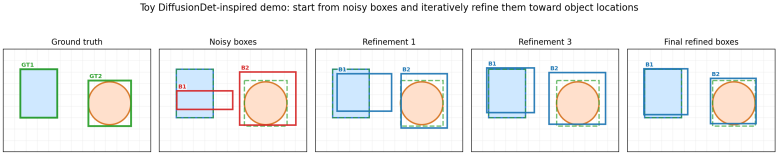
Application II: DiffusionDet training and inference

Training

1. Corrupt ground-truth boxes with Gaussian noise.
2. Combine image features, noisy boxes, and timestep information.
3. Predict denoised boxes and class scores.
4. Optimize detection losses after matching predictions to targets.

Inference

1. Initialize a set of random boxes.
2. Iteratively refine boxes and scores.
3. Filter or renew low-confidence boxes as the chain progresses.
4. Output the final set of detections.



Notebook demo: diffusion as iterative refinement of a structured output, not only an image generator.

Application III: very recent trends in AI

Diffusion Transformers (DiT) made transformer backbones central for latent-space diffusion.

Flow Matching gives a nearby continuous-time viewpoint that often leads to cleaner formulations and faster generation.

Large language / multimodal directions

- **LLaDA**: diffusion-style large language modeling,
- **Transfusion**: combine next-token prediction for text with diffusion over continuous modalities such as images.

A point

Autoregressive models still dominate text generation in practice, but diffusion-style ideas are increasingly relevant for multimodal and continuous-output problems.

Take-home messages

- Diffusion models are easiest to understand as **learned reverse denoisers**.
- DDPM gives the concrete discrete-time recipe students can implement from scratch.
- The score-based / SDE view is the cleanest unifying language for modern diffusion research.
- Conditional guidance, latent diffusion, and solver design make these models practical.
- Applications now span vision, time series, audio, inverse problems, robotics, and multimodal AI.

One sentence for the end

Diffusion works by learning a sequence of small corrections that transforms noise into data.

Suggested problems for students

Conceptual

- Why does sampling make diffusion a natural uncertainty model rather than only a point predictor?
- Why can stronger guidance improve condition fidelity but reduce diversity?

Analytical

- Derive the closed-form forward process $q(\boldsymbol{x}_t \mid \boldsymbol{x}_0)$ from repeated Gaussian perturbations.
- Starting from the ϵ-prediction update, explain why the reverse mean acts like a denoising estimator.

Implementation

- Modify the notebook stride in fast sampling and report the runtime-quality tradeoff.
- Change the inpainting mask or forecasting horizon and explain how the sample diversity changes.

Solution cues (full solutions in the appendix and notebook)

- **Sampling and uncertainty:** repeated runs produce multiple plausible outputs, so diffusion models represent a full distribution rather than a single point.
- **Guidance vs diversity:** stronger guidance pushes samples harder toward the condition, which often improves fidelity but suppresses alternative modes.
- **Closed-form forward process:** repeated Gaussian perturbations stay Gaussian; shrinkage accumulates into $\sqrt{\bar{\alpha}_t}$ and the total variance becomes $1 - \bar{\alpha}_t$.
- **Reverse mean:** with ϵ-prediction, subtracting the estimated noise is exactly a denoising move toward a cleaner sample.
- **Implementation exercises:** larger stride reduces runtime but usually lowers sample quality; weaker conditions or longer horizons should increase output diversity.

References

- Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. NeurIPS.
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., & Poole, B. (2021). *Score-Based Generative Modeling through Stochastic Differential Equations*. ICLR.
- Luo, C. (2022). *Understanding Diffusion Models: A Unified Perspective*.
- Rombach, R., Blattmann, A., Lorenz, D., Esser, P., & Ommer, B. (2022). *High-Resolution Image Synthesis with Latent Diffusion Models*. CVPR.
- Song, J., Meng, C., & Ermon, S. (2021). *Denoising Diffusion Implicit Models*. ICLR.
- Song, Y., Dhariwal, P., Chen, M., & Sutskever, I. (2023). *Consistency Models*. ICML.
- Peebles, W., & Xie, S. (2023). *Scalable Diffusion Models with Transformers*. ICCV.
- Rasul, K., Seward, C., Schuster, I., & Vollgraf, R. (2021). *TimeGrad*. ICML.
- Tashiro, Y., Song, J., Song, Y., & Ermon, S. (2021). *CSDI*. NeurIPS.
- Chen, S., Sun, P., Song, Y., & Luo, P. (2023). *DiffusionDet*. ICCV.
- Xu, J., Liu, S., Vahdat, A., et al. (2023). *ODISE*. CVPR.
- Ke, B., Obukhov, A., Huang, S., et al. (2024). *Marigold*. CVPR.

Appendix: detailed solutions I

Q1. Why does diffusion naturally model uncertainty?

- A deterministic predictor returns a single output $\hat{x}$ for each input, so uncertainty must be added separately.
- A diffusion model defines a *conditional distribution* through repeated sampling steps.
- Running the reverse chain multiple times with different random seeds yields different plausible outputs.
- The spread of these samples reflects aleatoric ambiguity in the condition and modeling uncertainty in the learned reverse process.

Appendix: detailed solutions II

Q2. Why can stronger guidance improve fidelity but reduce diversity?

- Guidance adds an extra term that pushes samples more strongly toward the condition c.
- This increases condition alignment, for example matching a prompt, class, or observation more closely.
- But a stronger push concentrates probability mass around a smaller subset of outputs, so distinct samples become more similar.
- Therefore, fidelity usually increases while diversity decreases; this is a tradeoff, not a contradiction.

Appendix: detailed solutions III

Q3. Derive the closed-form forward process.

Start from the single-step corruption

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_{t-1}) = \mathcal{N}\big(\sqrt{\alpha_t}\,\boldsymbol{x}_{t-1}, (1-\alpha_t)I\big), \qquad \alpha_t = 1-\beta_t.$$

Applying this recursively gives

$$\boldsymbol{x}_t = \sqrt{\alpha_t}\,\boldsymbol{x}_{t-1} + \sqrt{1-\alpha_t}\,\boldsymbol{\epsilon}_t.$$

Unrolling the recursion,

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sum_{s=1}^{t} \left(\sqrt{1-\alpha_s} \prod_{j=s+1}^{t} \sqrt{\alpha_j} \right) \boldsymbol{\epsilon}_s,$$

where $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$. Since the sum of independent Gaussians is Gaussian,

$$q(\boldsymbol{x}_t \mid \boldsymbol{x}_0) = \mathcal{N}\big(\sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0, (1-\bar{\alpha}_t)I\big).$$

So we can sample directly as:

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\,\boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\,\boldsymbol{\epsilon},$$
$$\boldsymbol{\epsilon} \sim \mathcal{N}(0, I).$$

Appendix: detailed solutions IV

Q4. Why does the reverse mean act like a denoising estimator?
For ϵ-prediction, the ancestral update uses

$$\boldsymbol{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t) \right) + \sigma_t \boldsymbol{z}.$$

The deterministic part is

$$\boldsymbol{\mu}_\theta(\boldsymbol{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\boldsymbol{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t) \right).$$

If $\boldsymbol{\epsilon}_\theta(\boldsymbol{x}_t, t) \approx \boldsymbol{\epsilon}$, then substituting the forward model

$$\boldsymbol{x}_t = \sqrt{\bar{\alpha}_t}\, \boldsymbol{x}_0 + \sqrt{1-\bar{\alpha}_t}\, \boldsymbol{\epsilon}$$

shows that the term subtracted from $\boldsymbol{x}_t$ removes the noise contribution. Hence $\boldsymbol{\mu}_\theta$ points toward a cleaner estimate of the sample at the previous time step. The stochastic term $\sigma_t \boldsymbol{z}$ keeps the chain probabilistic and preserves sample diversity.

Appendix: detailed solutions V

Q5. How are ϵ-, x_0-, and v-prediction related?

- ϵ-prediction learns the injected noise directly.
- x_0-prediction learns the clean sample that generated the noisy observation.
- v-prediction is a linear reparameterization that balances both views and often improves stability across noise levels.
- They are different parameterizations of the same denoising task, not unrelated objectives.

Appendix: detailed solutions VI

Q6. What do the two terms in Langevin dynamics do?

$$\boldsymbol{x}_{k+1} = \boldsymbol{x}_k + \eta \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}_k) + \sqrt{2\eta}\, \boldsymbol{z}_k.$$

- The drift term $\eta \nabla_{\boldsymbol{x}} \log p(\boldsymbol{x}_k)$ moves the sample toward higher-density regions.
- The noise term $\sqrt{2\eta}\, \boldsymbol{z}_k$ prevents collapse to a single mode and enables stochastic exploration.

Q7. What happens when fast-sampling stride increases?

- Runtime decreases because fewer reverse steps are executed.
- Sample quality may degrade because the solver skips intermediate denoising corrections.
- The tradeoff is application dependent: a moderate reduction in steps often saves time with only small quality loss.

Appendix: detailed solutions VII

Q8. What happens if the inpainting mask or forecast horizon changes?

- Larger missing regions or longer horizons increase uncertainty because fewer observations constrain the output.
- As a result, sample diversity should increase.
- Smaller masks or shorter horizons reduce ambiguity, so samples become more concentrated.

Q9. Why is DiffusionDet a diffusion example rather than only a detector?

- It starts from noisy boxes and iteratively refines them.
- The detector output is therefore produced by a learned reverse-refinement process.
- This demonstrates that diffusion is a general iterative inference principle for structured outputs, not only an image-generation tool.